Inheritance in Java:

Inheritance in Java is a **core** **OOP** concept that **allows** a class to **acquire properties** and **behaviors** from **another** class. It helps in **creating** a **new class** from an **existing class**, promoting **code reusability** and **better organization**.

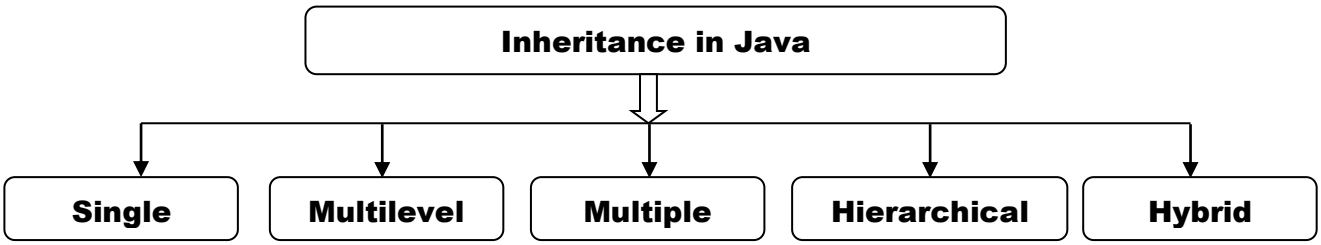
- **Subclass** can **reuse** the **fields** and **methods** of the **parent class** without **rewriting** the **code**
- Subclass can **add** its **own fields** and **methods** or **modify existing** ones to **extend** functionality.

```
   // Parent class
class Animal
{
    void sound()
    {
        System.out.println("Animal makes a sound");
    }
}
      // Child class
class Dog extends Animal
{
    void sound()
    {
        System.out.println("Dog barks");
    }
}
      // Child class
class Cat extends Animal
{
    void sound()
    {
        System.out.println("Cat meows");
    }
}
      // Child class
class Cow extends Animal
{
    void sound()
{
        System.out.println("Cow moos");
    }
}
      // Main class
public class Sample
{
    public static void main(String[] args)
{
        Animal A;
        A = new Dog();
        A.sound();
        A = new Cat();
        A.sound();
        A = new Cow();
        A.sound();
    }
}
```

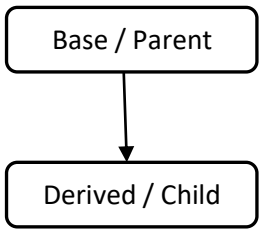
Output:

```
Dog barks
Cat meows
Cow moos
```

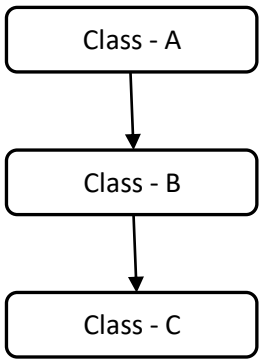
Types Of Inheritance in Java:



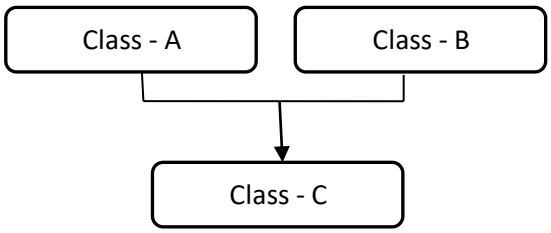
Single



Multilevel



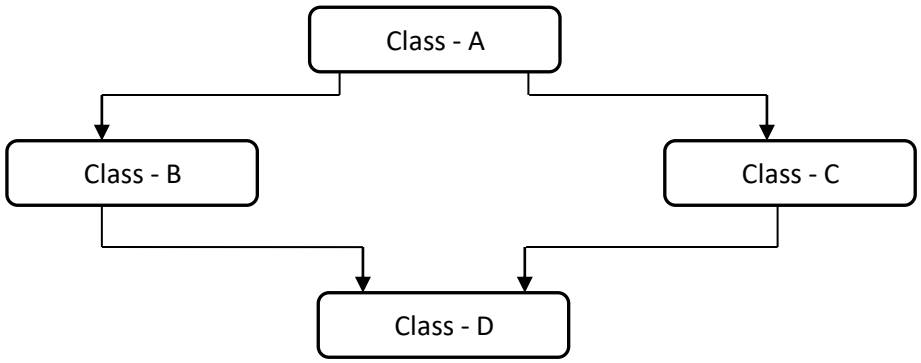
Multiple



Hierarchical



Hybrid



Single Inheritance:

In **Single Inheritance**, a **sub-class** is **derived** from **only one super or base class**. It **inherits** the **properties** and **behavior** of a **single-parent class**. Sometimes, it is also known as **Simple Inheritance**.

```
class Vehicle
{
    Vehicle()
    {
        System.out.println ("This is a Vehicle");
    }
}
class Car extends Vehicle
{
    Car()
    {
        System.out.println("This Vehicle is Car");
    }
}
public class Test
{
    public static void main(String[] args)
    {
        Car C = new Car();
    }
}
```

Output:

```
This is a Vehicle
This Vehicle is Car
```

Multilevel Inheritance:

In **Multilevel Inheritance**, a **derived class** will be **inheriting** a **base class** and as well as the **derived class** also **acts** as the **base class** for **other classes**.

```
class Vehicle
{
    Vehicle ()
{
        System.out.println ("This is a Vehicle");
    }
}
class FourWheeler extends Vehicle
{
    FourWheeler ()
    {
        System.out.println ("4 Wheeler Vehicles");
    }
}
class Car extends FourWheeler
{
    Car ()
{
        System.out.println ("This Vehicle is a Car");
    }
}
```

```
public class Sample
{
    public static void main(String[] args)
       {
        Car C = new Car ();
       }
}
```

Output:

```
This is a Vehicle
4 Wheeler Vehicles
This Vehicle is a car
```

Hierarchical Inheritance:

In **Hierarchical Inheritance**, **more than one subclass** is **inherited** from a **single base class**. I.e. more than one **derived classes** are created from a **single base class**.

```
class Vehicle
{
    Vehicle()
    {
        System.out.println ("This is a Vehicle");
    }
}
class Car extends Vehicle
{
    Car()
    {
        System.out.println ("This Vehicle is Car");
    }
}
class Bus extends Vehicle
{
    Bus()
    {
        System.out.println ("This Vehicle is Bus");
    }
}
public class Test
{
    public static void main(String[] args)
    {
        Car C = new Car ();
        Bus B = new Bus ();
    }
}
```

Output:

```
This is a Vehicle
This Vehicle is Car
This is a Vehicle
This Vehicle is Bus
```

Multiple Inheritances:

In **Multiple inheritances**, one class can have more than one **super class** and **inherit Features** from all **parent classes**.

Note: that Java **does not support multiple inheritances** with classes. In Java, we can achieve multiple inheritances only through **Interfaces**.

```
interface LandVehicle
{
    default void landInfo()
    {
        System.out.println("This is a Land Vehicle");
    }
}
interface WaterVehicle
{
    default void waterInfo()
{
        System.out.println("This is a Water Vehicle");
    }
}
class AmphibiousVehicle implements LandVehicle, WaterVehicle
 {
    AmphibiousVehicle()
    {
        System.out.println("This is an Amphibious Vehicle");
    }
}
public class Test
{
    public static void main(String[] args)
    {
        AmphibiousVehicle AV = new AmphibiousVehicle();
        AV.waterInfo();
        AV.landInfo();
    }
}
```

Output:

```
This is a Amphibious Vehicle
This is a Water Vehicle
This is a Land Vehicle
```

Interface in Java:

An **interface** in Java is a **blueprint** that defines a **set** of **methods** a class must implement without **providing full implementation** details. It **helps achieve abstraction** by **focusing** on what a class should do rather than how it does it. **Interfaces** also **support multiple inheritances** in Java.

```
interface Sample
{
        final int a = 10;    //interface can have only final fields
        void display();     //interface can have only abstract methods
}
```

Hybrid Inheritance:

It is a **mix** of **two** or **more** of the above **types** of **inheritance**. In **Java**, we can **achieve hybrid inheritance** only through **Interfaces** if we want to involve **multiple inheritances** to implement Hybrid inheritance.

```
      // Superclass
class Vehicle
{
    void vehicleType()
    {
        System.out.println("This is a Vehicle");
    }
}
      // Interface for fare
interface Fare
{
    default void fareInfo()
    {
        System.out.println("Fare Information");
    }
}
      // Subclass 1: Single inheritance + multilevel
class Car extends Vehicle
{
    void carType()
    {
        System.out.println("This is a Car");
    }
}
      // Subclass 2: Hybrid inheritance (extends class + implements interface)
class Bus extends Vehicle implements Fare
{
    void busType()
    {
        System.out.println("This is a Bus");
    }
}

public class Sample
{
    public static void main(String[] args)
    {
        Car C = new Car();
        C.vehicleType(); // inherited from Vehicle
        C.carType();     // specific to Car

        Bus B = new Bus();
        B.vehicleType(); // inherited from Vehicle
        B.busType();     // specific to Bus
        B.fareInfo();    // from Fare interface
    }
}
```

Output:

```
This is a Vehicle
This is a Car
This is a Vehicle
This is a Bus
Fare Information
```

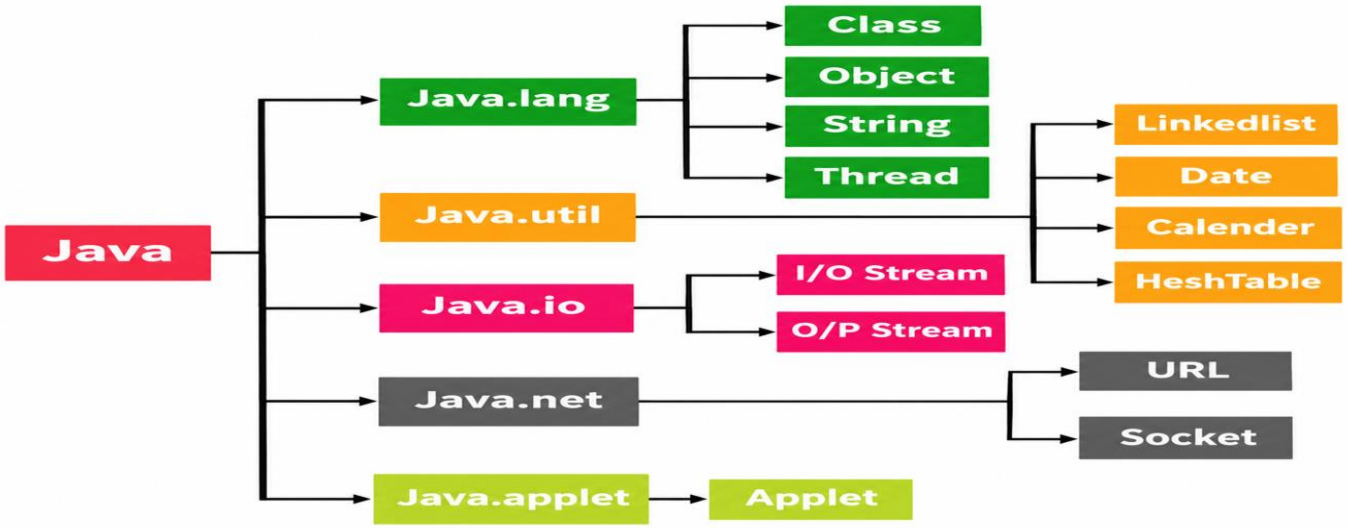
Packages in Java:

Packages in Java is the way of **grouping variety** of **class** and/or **interfaces together**. The **Grouping** is usually **done according** to **functionality**. In fact, packages acts as "**Containers**" for **Classes/Interfaces**.

Benefits of using Packages in Java

- **Organize** related **classes** and **interfaces**.
- **Avoid** naming **conflicts**.
- Provide **access control**.
- **Improve** code **maintainability**.
- Promote **code reuse**.
- Make large **applications easier** to **manage**.
- Support **modular application design**.

Built-in API Packages in Java:



java.lang: Contains foundational classes essential to the design of the language, such as Object, String, Math, and primitive type wrappers (Integer, Double). **This package is automatically imported** into every Java program.

java.util: Provides the Java Collections Framework (e.g., Array List, Hash Map), date/time facilities, internationalization, and miscellaneous utility classes like Scanner and Random.

java.io: Provides classes for system input and output through data streams, serialization, and the file system (e.g., File, Input Stream, and Buffered Reader).

java.net: Provides classes for implementing networking applications, enabling socket connections, URL management, and HTTP requests.

java.applet: Contains classes for creating and implementing applets.

Package Declaration:

Package can be **declared** in **Java** by using the **keyword package** followed by **user-defined package name** as follows.

```
package Package_Name;

Ex: package sample;
```

Creating Package:

In Java **Packages** are created by using the following simple steps.

- Create a **folder (package)** by the name **package-name** in the java working **bin** directory.
- Declare the package at the beginning of the file using the form **package package-name;** inside the **folder**
- Define the **class** that is to be **put** in the **package** and **declare** it as **public**.
- After successful completion of defining of class, get back out of the **folder (package).**
- Create **Java Source File** within the **bin** directory.
- **Import** the **newly created folder** (package) at the **beginning** of **java source file**.
- **Compile** and **run** the **java file**, which will also **import** and **executes** the **created package**.

Accessing Package:

Package in java can be **accessed** by using the **import statement** and **fully qualified package name** as follows.

Example:

import **java**.**io**.*;
import **java**.**util**.**Scanner**;
import **sample;**

Using a Package:

Packages in Java are used in a very simple way as follow.

```
package sample
public class A
{
	Public void display()
	{
		System.out.println("This is Class-A");
	}
}

import sample;
public class B
{
	Public void display()
	{
		System.out.println("This is Class-B");
	}
}
```

Package CVR under the directory D:\Program Files\Java\jdk1.8.0_60\bin\CVR\Student.java

```
package CVR;
public class Student
{
      int Regno;
      String Name;

      public void getdata (int r, String s)
      {
            Regno = r;

            Name = s;

      }

      public void putdata ()

      {

            System.out.println ("Regno of Student is=" + Regno);

            System.out.println ("Name of Student is=" + Name);

      }

}
```

Type Program under D:\Program Files\Java\jdk1.8.0_60\bin\PackageDemo.java

```
import CVR.*;
class PackageDemo
 {

      public static void main(String args[])
      {

            Student S = new Student();
            S.getdata(123,"Akshara");
            S.putdata();
      }

}
```

Compile and **Execute** the above **program**, which will **imports** the **user defined package CVR**

Java Errors:

Even experienced Java developers make mistakes. The key is learning how to **spot** and **fix** them!

Here, we cover common errors & helpful debugging tips to help & understand what's going wrong and how to fix it.

Types of Errors in Java:

Compile-Time Error:	Detected by the Compiler. Prevents code from running.
Run-Time Error:	Occurs while the program is running. Often causes crashes.
Logical error:	Code runs but gives incorrect results. Hardest to find.

1. Common Compile-Time Errors:

Compile-time errors occur when the program cannot compile due to syntax or type issues.

Here are some examples:

Missing Semicolon

```
int x = 5
System.out.println(X);
```

```
error: ';' expected
```

Undeclared Variables

```
System.out.println(Name);
```

```
cannot find symbol
  symbol: variable myVar
```

Mismatched Types

```
int x = "Hello";
```

```
incompatible types: String cannot be converted to int
```

2. Common Run-Time Errors:

Occur when the program compiles but crashes or behaves unexpectedly.

Here are some examples:

Division by Zero

```
int x = 10;
int y = 0;
int Result = x / y;
System.out.println (Result);
```

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

Array Index Out Of Bounds

```
int[ ] Numbers = { 10, 20, 30};

System.out.println (Numbers[5]);
```

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
```

3. **Logical Errors:** happens when the code runs, but the result is not what we thought.

Here are some examples:

```
int x = 10;
int y = 2;
int Result = x - y;
System.out.println ("x + y =" +Result);
```

```
x + y = 8
```

Expected Result: **12**

Good Habits to Avoid Errors:

- Use meaningful variable names
- Read the error message carefully. What line does it mention?
- Check for missing semicolons or braces
- Look for typos in variable or method names

Exceptions in Java:

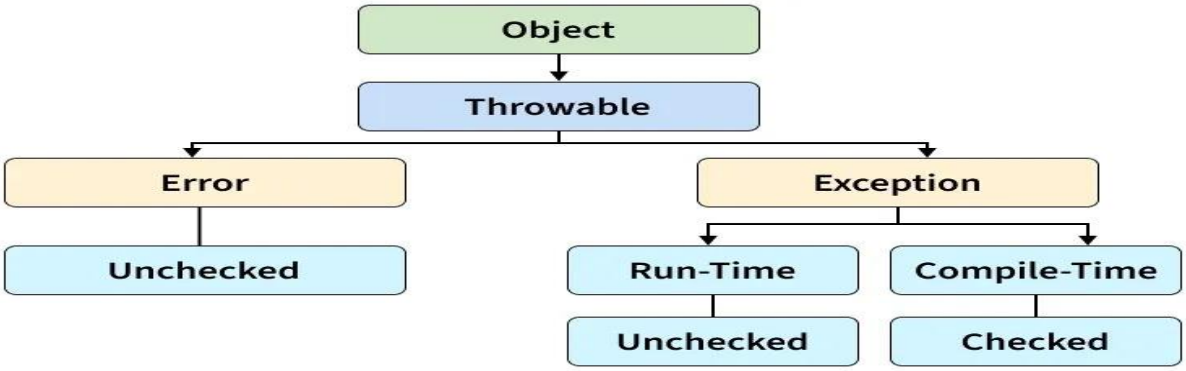
Exception handling in Java is a **mechanism** used to **handle exceptional conditions** that occur **during program execution**. It helps **prevent** the **program** from **terminating unexpectedly** and allows **developers** to **provide appropriate recovery** or **error-handling logic**.

- **Separates error-handling** code from normal **program logic**.
- Helps **maintain** the **normal flow** of program execution.
- Supports handling of both **checked** and **unchecked** exceptions.

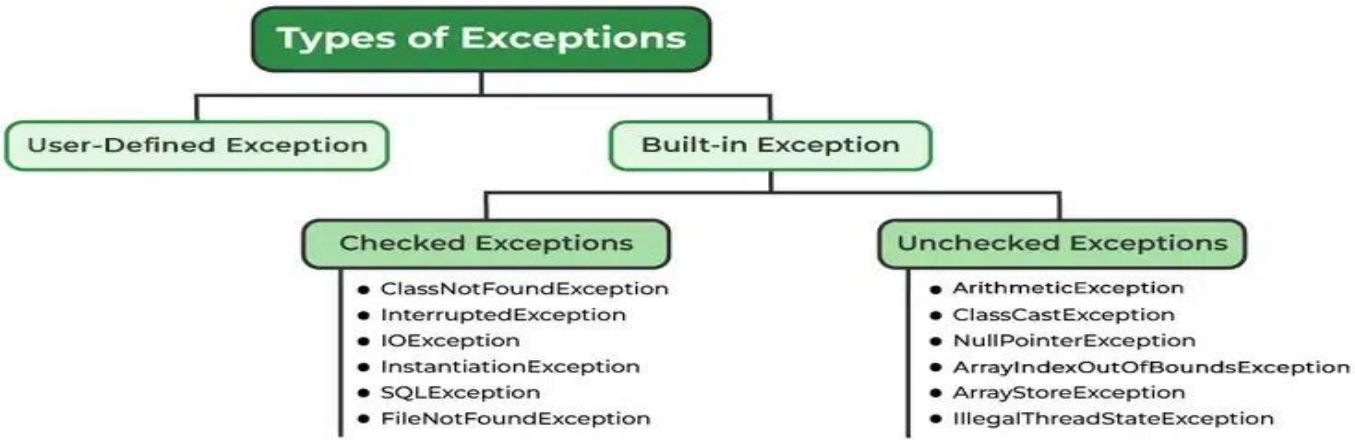
Java Exception Hierarchy:

In Java, all **exceptions** and **errors** are **subclasses** of the **Throwable** class. It has **two** main branches.

1. **Exception.**
2. **Error**



Types of Java Exceptions: Java defines **several types** of **exceptions** that relate to its **various class libraries**. Java also allows **users** to **define** their **exceptions**.



User-defined Exceptions: Sometimes, the **built-in exceptions** in Java are not able to **describe** a certain **situation**. In such cases, **users** can also **create exceptions**, which are called "**user-defined Exceptions**".

Built-in Exceptions: **Built-in Exceptions** are **pre-defined exception** classes provided by **Java** to handle common errors during program execution. There are two type of **built-in exception** in **java**.

- **Checked Exception:** These exceptions are checked at compile time, forcing the programmer to handle them explicitly.
- **Unchecked Exception:** These exceptions are checked at runtime and do not require explicit handling at compile time.

Basic try-catch: The **try block** contains **code** that might **throw** an **exception**; the **catch block** handles the **exception** if it occurs.

```
class Sample
{
    public static void main(String[] args)
      {
            int n = 10;         int m = 0;
            try
            {
                int ans = n / m;
                System.out.println("Answer: " + ans);
            }
            catch (ArithmeticException e)
             {
                System.out.println("Error: Division by 0!");
             }
    }
}
```

Output: Error: Division by 0!

Explanation: The division operation throws an **ArithmeticException**. The matching catch block catches the exception and prints an appropriate message instead of allowing the program to terminate abruptly.

Finally Block: The **finally block** is used to **execute cleanup code** after **try** and **catch**. It **normally executes** whether an **exception occurs** or **not**. Finally may **not execute** in **cases** like:

1. **System**. **exit**()
2. **JVM crash**
3. **infinite loop** before finally

```
class FinallyExample
{
    public static void main(String[] args)
      {
        int[] numbers = {1, 2, 3};
        try
        {
            System.out.println(numbers[5]);
        }
       catch (ArrayIndexOutOfBoundsException e)
       {
            System.out.println("Exception caught.");
        }
       finally
        {
            System.out.println("Finally block executed.");
        }
        System.out.println("Program continues...");
    }
}
```

Output:

```
Exception caught.
Finally block executed.
Program continues...
```

throw Keyword: Used to explicitly throw a single exception.

```
class Demo
{
	static void checkAge(int age)
	{
		if (age < 18)
		{
			throw new IllegalArgumentException("Age must be 18 or above");
		}
	}
	public static void main(String[] args)
	{
		checkAge(15);
	}
}
```

```
Output:	Exception in thread "main" java.lang.IllegalArgumentException: Age must be
	18 or above at Demo.checkAge(Demo.java:5) at Demo.main(Demo.java:11)
```

throws Keyword: The throws keyword is used in a method declaration to indicate that the method may throw one or more exceptions.

```
import java.io.*;
class Demo
{
	static void readFile(String fileName)
		throws IOException
		{
			try (FileReader file = new FileReader(fileName))
			{
				int data;
				while ((data = file.read()) != -1)
				{
					System.out.print((char) data);
				}
			}
		}
public static void main(String[] args)
{
	try
	{
		readFile("test.txt");
	}
	catch (IOException e)
	{
		System.out.println("File not found or error reading file: " + e.getMessage());
	}
	System.out.println("\nProgram continues after file operation.");
    }
}
```

```
Output:	File not found or error reading file: test.txt (No such file or directory)
	Program continues after file operation.
```

Nested try-catch:

In Java, you can place one try-catch block inside another to handle exceptions at multiple levels.

```
public class NestedTryExample
{
    public static void main(String[] args)
      {
        try
             {
                   System.out.println("Outer try block");
                   try
                    {
                          int a = 10 / 0;
                    }
                    catch (ArithmeticException e)
                    {
                          System.out.println("Inner catch: " + e);
                    }
                           String str = null;
                           System.out.println(str.length());
              }
                    catch (NullPointerException e)
                    {
                          System.out.println("Outer catch: " + e);
                    }
      }
}
```

Output:

```
Outer try block
Inner catch: java.lang.ArithmeticException: / by zero
Outer catch: java.lang.NullPointerException: Cannot invoke "String.length()"
because "<local1>" is null
```

Explanation: The inner try-catch handles the ArithmeticException. Later, the outer try encounters a NullPointerException, which is handled by the outer catch.

Handling Multiple Exceptions:

We can handle multiple type of exceptions in Java by using multiple catch blocks, each catching a different type of exception.

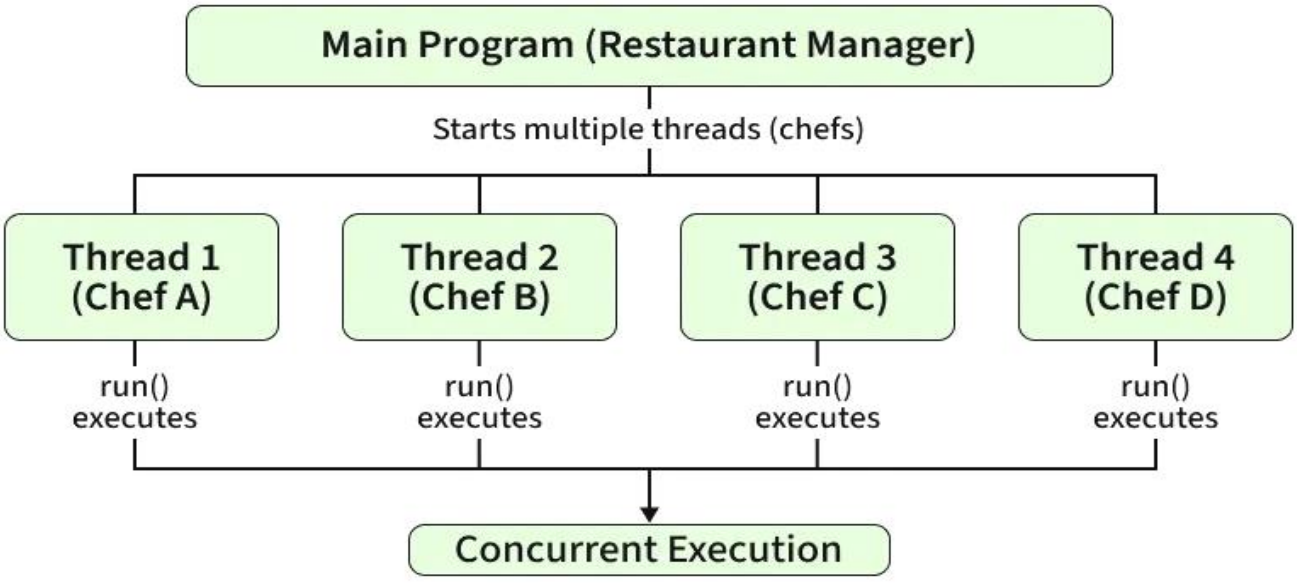
```
try
{
      } catch (ArithmeticException e) {      }

      } catch(ArrayIndexOutOfBoundsException e)     {      }

      }catch(NumberFormatException e)  {      }
}
```

Error/Exception	Description
ArithmeticError	Occurs when a numeric calculation goes wrong
ArrayIndexOutOfBoundsException	Occurs when trying to access an index number that does not exist in an array
ClassNotFoundException	Occurs when trying to access a class that does not exist
FileNotFoundException	Occurs when a file cannot be accessed
InputMismatchException	Occurs when entering wrong input (e.g. text in a numerical input)
IOException	Occurs when an input or output operation fails
NullPointerException	Occurs when trying to access an object reference that is null
NumberFormatException	Occurs when it is not possible to convert a specified string to a numeric type
StringIndexOutOfBoundsException	Occurs when trying to access a character in a String that does not exist

Multithreading in Java: **Multithreading** in Java is a **feature** that **enables** a **program** to **run multiple threads** simultaneously, allowing tasks to execute in **parallel** and **utilize** the **CPU** more **efficiently**. Thread is a **lightweight**, **independent unit** of execution inside a **program** (Process).



Different Ways to Create Threads:

Threads can be created by using two Mechanisms

1. Extending the Thread class:

We **create** a **class** that **extends Thread** and override its **run() method** to define the task. Then, we make an **object** of this **class** and **call start(),** which automatically calls **run()** and begins the **thread's execution**.

Example: Restaurant Kitchen (Extending Thread)

```
class CookingTask extends Thread
{
    private String task;
    CookingTask(String task)
    {
        this.task = task;
    }
    public void run()
    {
       System.out.println(task + " is being prepared by " +Thread.currentThread().getName());
    }
}

public class Restaurant
{
    public static void main(String[] args)
       {
        Thread T1 = new CookingTask("Pasta");
        Thread T2 = new CookingTask("Salad");
        Thread T3 = new CookingTask("Dessert");
        Thread T4 = new CookingTask("Rice");

        T1.start();
        T2.start();
        T3.start();
        T4.start();
      }
}
```

Output:

```
Rice is being prepared by Thread-3
Pasta is being prepared by Thread-0
Dessert is being prepared by Thread-2
Salad is being prepared by Thread-1
```

Explanation:

- We created multiple threads (t1–t4) using the CookingTask class.
- Each thread represents a dish being prepared.
- Calling start() creates a new thread with its own call stack and internally invokes the run() method. This allowsthreads to run concurrently with the main thread and each other.

2. Implementing the Runnable Interface:

We create a new class which implements java.lang. Runnable interface and define the run() method there. Then we instantiate a Thread object and call start() method on this object.

Example: Restaurant Kitchen (Runnable Interface)

```
class CookingJob implements Runnable
{
    private String task;
    CookingJob(String task)
    {
        this.task = task;
    }
    public void run()
    {
       System.out.println(task + " is being prepared by " +Thread.currentThread().getName());
    }
}

public class RestaurantRunnable
{
    public static void main(String[] args)
    {
        Thread T1 = new Thread(new CookingJob("Soup"));
        Thread T2 = new Thread(new CookingJob("Pizza"));
        Thread T3 = new Thread(new CookingJob("Burger"));

        T1.start();
        T2.start();
        T3.start();
   }
}
```

Output:

```
Burger is being prepared by Thread-2
Pizza is being prepared by Thread-1
Soup is being prepared by Thread-0
```

Explanation:

- Cookingob implements Runnable and overrides run().
- We pass a Runnable object to the Thread constructor.
- Calling start() starts a new thread and invokes its run() method. The thread runs independently, and itsexecution order is not guaranteed, so the output may appear in a different order each time.

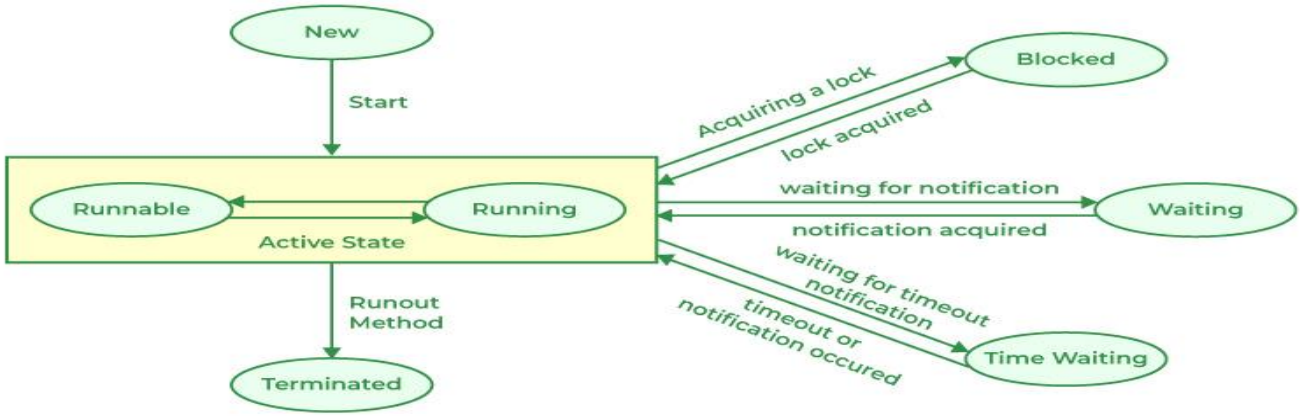
Advantages of Multithreading:

- **Improved Performance:** Multiple tasks can run simultaneously, reducing execution time.
- **Efficient CPU Utilization:** Threads keep the CPU busy by running tasks in parallel.
- **Responsiveness:** Applications (like GUIs) remain responsive while performing background tasks.
- **Resource Sharing:** Threads within the same process share memory and resources, avoiding duplication.
- **Better User Experience:** Smooth execution of tasks like file downloads, animations, and real-time updates.

Thread Life Cycle:

The **lifecycle** of a **thread** in **Java describes** the **different states** a **thread goes through** from its **creation** until **its execution** is **completed**.

- **Thread** moves **between** different **states** during its **lifetime**.
- **Thread** methods such as **start** (), **sleep** (), **wait** (), and **join** () can cause **state transitions**.
- **Understanding** thread states **helps** in **debugging** and **managing multithreaded** applications.



1. New: A thread is in the **New** state when its Thread object has been created but the start() method has not yet been called.

2. Runnable: Thread state for a **runnable** thread. A thread in the runnable state is executing in the Java virtual machine but it may be waiting for other resources from the operating system such as a processor.

3. Running: in this state the thread is executing (running) a process in their critical section.

4. Blocked: A thread is in the **blocked** state when it is waiting to acquire a lock that is currently held by another thread.

5. Waiting: Thread state for a **waiting** thread. A thread is in the waiting state due to calling one of the following methods:

6. Timed Waiting: Thread state for a waiting thread with a specified waiting time. A thread is in the timed waiting state due to calling one of the following methods with a specified positive waiting time:

7. Terminated: Thread state for a **terminated** thread. The thread has completed execution.

Thread Methods:

start ()	**sleep ()**	**wait ()**	**join ()**	**interrupt ()**
getState ()	**resume ()**	**notify ()**	**run()**	**yield()**

```
import java.lang.Thread;
class A extends Thread
     {
     public void run()
         {
             System.out.println ("Thread A Started:");
            for (int i=1;i<=5;i++)
             {
             System.out.println ("\t From Thread A:i="+i);
             }
            System.out.println ("Exit From Thread A:");
         }
     }
class B extends Thread
     {
         public void run()
         {
             System.out.println ("Thread B Started:");
             for(int j=1;j<=5;j++)
             {
                 System.out.println ("\t From Thread B:j="+j);
             }
            System.out.println ("Exit From Thread B:");
         }
     }
class C extends Thread
     {
         public void run()
         {
             System.out.println ("Thread C Started:");
             for(int k=1;k<=5;k++)
             {
                 System.out.println ("\t From Thread C:k="+k);
             }
            System.out.println ("Exit From Thread C:");
         }
     }
class ThreadTest
     {
         public static void main(String args[])
         {
           new A().start();
           new B().start();
           new C().start();
        }
     }
```

Thread Synchronization:

Thread Synchronization is used to coordinate and ordering of the execution of the threads in a multi-threaded program. There are two types of thread synchronization are mentioned below:

```
class TicketBooking
{
    // Shared resource (available tickets)
    private int availableTickets = 10;

    // Synchronized method for booking tickets
    public synchronized void bookTicket(int tickets)
       {

        if (availableTickets >= tickets)
        {

            availableTickets -= tickets;
            System.out.println("Booked " + tickets + " tickets, Remaining tickets: "
                               + availableTickets);
        }
        else
       {
            System.out.println("Not enough tickets available to book "+ tickets);
        }
    }

    public int getAvailableTickets()
    {

        return availableTickets;
    }
}

public class Geeks
{
    public static void main(String[] args)
    {

        // Shared resource
        TicketBooking booking = new TicketBooking();

        // Thread 1 to book tickets
        Thread t1 = new Thread(() ->
         {
            for (int i = 0; i < 2; i++)
             {

                // Trying to book 2
                // tickets each time
                booking.bookTicket(2);

                try
                {
                    // Simulate delay
```

```
                    Thread.sleep(50);
                }
                catch (InterruptedException e)
                {
                    e.printStackTrace();
                }
            }
        });

        // Thread 2 to book tickets
        Thread t2 = new Thread(() ->
        {
            for (int i = 0; i < 2; i++)
             {

                // Trying to book 3
                // tickets each time
                booking.bookTicket(3);

                try{

                    // Simulate delay
                    Thread.sleep(40);
                }
                catch (InterruptedException e)
                {
                    e.printStackTrace();
                }
            }
        });

        // Start both threads
        t1.start();
        t2.start();

        // Wait for threads to finish
        try {
            t1.join();
            t2.join();
        }
        catch (InterruptedException e)
        {
            e.printStackTrace();
        }

        // Print final remaining tickets
        System.out.println("Final Available Tickets: "+ booking.getAvailableTickets());
    }
}
```